# JWT 게시판 구현 보고서

## 01 서비스 소개와 구현 범위

**학번** 202446609 이름: 곽우현 제출일: 2026-10-28

**서비스 이름** 맛집 후기 게시판

**기술 조합과 버전** React 19.3.0 + Vite 8.3.4 / Express 5.3.0 (jsonwebtoken 9.0.3, sql.js 1.14.2, dotenv 18.0.7) / Windows 10.0.26300, Node.js v24.21.0, PowerShell 7.6.9, Chrome 156

**GitHub 저장소 URL** https://github.com/zoe6rain/jwt-matjip-board

**소스 코드 기준** main 브랜치 / 코드 구현 기준 커밋 a5c2f28 (최종 제출 기준 커밋 SHA 는 LMS 에 제출)

다녀온 맛집의 후기를 남기고 공유하는 게시판입니다. 후기를 찾아보는 사람은 로그인 없이 목록과 상세를 읽을 수 있고, 후기를 쓰려는 사용자는 로그인한 뒤 작성합니다. 글을 쓴 사람만 자기 후기를 수정하거나 삭제할 수 있습니다. 수업 시간에 JWT 로그인을 React 와 Express 조합으로 배웠고, 수업 코드(06 JWT 로그인)를 그대로 따라 구현할 수 있어서 이 조합을 선택했습니다. 제공 템플릿에서 바꾼 범위는 서버의 backend-express/src/auth.js(토큰 발급·검증·작성자 검사)와 프론트의 frontend-react/src/api.js, main.jsx 의 로그인 연결·토큰 전송·화면 처리이고, 라우트·DB(store.js)·계정(users.js)·검사 프로그램은 제공 코드를 그대로 썼습니다.

| 기능 | 완료 여부 | 관련 파일 | 비고 또는 제한 |
|---|---|---|---|
| 공개 목록과 상세 | 완료 | backend app.js(GET /api/posts), frontend api.js list·detail | 로그인 없이 조회 (T01) |
| 로그인과 JWT 발급 | 완료 | auth.js issueToken, app.js 로그인 라우트, api.js login | HS256, 기본 TTL 90 초, 예시·잘못된 키는 서버 시작 거부 |
| 인증과 내 정보 조회 | 완료 | auth.js currentUserId, app.js /api/me | 토큰 없음·변조·만료를 401 코드로 구분 |
| 본인 글 작성·수정·삭제 | 완료 | app.js 글 라우트 + auth.js assertOwner, api.js canEdit | 작성자 검사는 서버(403), 화면은 버튼만 숨김 |
| 만료·로그아웃·오류 표시 | 완료 | api.js request·messages, main.jsx | 401 이면 로그인 상태 해제, 새로고침해도 해제 |
| DB 데이터 유지 | 완료 | store.js (sql.js → data/board.sqlite) | 서버 재시작 후 글 유지 (T15) |

# 구조와 변경 내역

브라우저 화면(React)은 /api 로 요청을 보내고, Vite 프록시가 이를 8000 번 Express 서버로 전달합니다. 서버는 로그인 요청에서 JWT 를 만들어 돌려주고, 글을 바꾸는 요청에서는 Authorization 헤더의 토큰을 검증한 뒤 SQLite 파일(data/board.sqlite)에 저장합니다. 토큰은 화면의 api.js 변수(메모리)에만 있고 localStorage 에는 저장하지 않습니다.

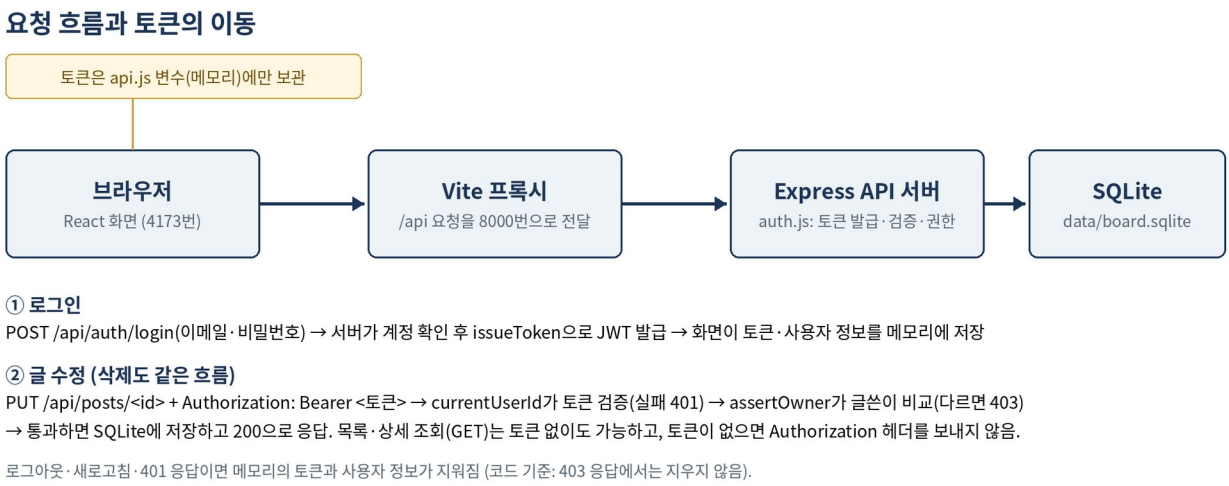



그림 1. 요청 흐름과 토큰의 이동

| 변경한 파일과 함수 | 제공 코드의 원래 역할 | 내가 변경한 내용 | 변경 이유 |
|---|---|---|---|
| backend-express/src/auth.js<br>설정(맨 위)·issueToken | 임시 코드가 501 NOT_IMPLEMENTED 를 던짐 | .env 의 JWT_SECRET 을 64 자리 16 진수로 검사(아니면 서버 시작 거부)하고 32 바이트 키로 변환. issueToken 은 HS256 JWT(sub=사용자 id 문자열, iss, aud, exp)를 문자열로 반환 | BE-01. 약한·예시 키로 서버가 켜지면 안 됨. 응답은 app.js 가 { accessToken }으로 감쌈<br>작성 출처: AI 초안(수업 06 JWT 28 쪽 기준) → 본인이 붙여넣고 실행해 확인 |
| auth.js<br>readBearer·validClaims·currentUserId | 데모 ID 1 을 항상 반환 | 헤더에서 토큰 추출 → jwt.verify(알고리즘·iss·aud·만료) → sub·exp 모양 확인 → 실제 사용자 확인 → id 반환. 실패는 401(TOKEN_REQUIRED·INVALID_TOKEN·TOKEN_EXPIRED) | BE-02. 누구의 요청인지는 토큰으로만 정함<br>작성 출처: AI 초안(수업 06 JWT 35·37·38·40 쪽 기준) → 본인 실행 확인 |
| auth.js<br>assertOwner | 검사 없음(누구나 통과) | 로그인 사용자 id 와 글쓴이 id 가 다르면 403 FORBIDDEN | BE-03. 다른 사람 글 수정·삭제 방지<br>작성 출처: AI 초안 → 본인 실행 확인(1·2 차 실패 후 통과, 5 절 사례 2) |

| 변경한 파일과 함수 | 제공 코드의 원래 역할 | 내가 변경한 내용 | 변경 이유 |
|---|---|---|---|
| frontend-react/src/api.js<br>login · authorizationHeaders · request · canEdit | login 은 오류를 던지는 임시 코드, 헤더는 빈 객체, canEdit 는 항상 true | login 이 토큰·사용자를 메모리 session 에 저장, 토큰이 있을 때만 Authorization: Bearer 전송, 401 이면 session 삭제, 오류 code 별 안내 문구, canEdit 는 id 비교 | FE-01~04<br>작성 출처: AI 초안 → 본인 이 붙여넣고 화면에서 확인 |
| frontend-react/src/main.jsx<br>index.html | 시작 코드 안내 배너와 STARTER 제목 | 안내 배너를 실제 설명으로 교체, 서비스 제목을 '맛집 후기 게시판'으로 변경, 404 응답이면 목록 새로고침 | 체크리스트의 데모 배너 교체, 주제 반영<br>작성 출처: AI 안내 → 본인 수정 |
| backend-express/src/app.js<br>(시작 메시지 한 군데) | 서버 시작 시 '인증 TODO 미완성' 문구 출력 | 시작 문구만 교체(라우트 코드는 제공 그대로 유지) | 미완성 안내가 남지 않게 정리<br>작성 출처: AI 안내 → 본인 수정 |
| README.md (저장소 루트) | 제공 양식 | 서비스 소개·실행 방법·테스트 요약·AI 활용 요약 작성 | 제출 요구 사항<br>작성 출처: AI 초안 → 본인 저장 |

제공 코드를 그대로 유지한 파일: app.js 의 라우트·입력 검사, store.js, users.js, errors.js, style.css, vite.config.js, tools/check_api.py. 추가한 보조 파일: 실행에 필요한 backend-express/.env(비밀 키, Git 에는 올리지 않음)와 docs/evidence/(테스트 캡처).

## 로컬 실행 기록

환경: Windows(빌드 10.0.26300), PowerShell 7.6.9, Node.js v24.21.0, Chrome 156. 작업 폴더 D:\JWT-Assignment 에서 backend-express 는 npm ci → .env 준비(.env.example 을 복사해 JWT_SECRET 에 node 로 만든 64 자리 난수를 넣음, 값은 기록하지 않음) → npm start, frontend-react 는 npm ci → npm run build → npm run preview 로 실행했습니다. 빌드는 vite v8.3.4 에서 '16 modules transformed, built in 536ms'로 성공했고(그림 B01), preview 는 http://127.0.0.1:4173 에서 열렸습니다(B02). SQLite 파일은 서버를 켤 때 backend-express/data/board.sqlite 로 자동 생성되며 Git 에는 올라가지 않습니다(.gitignore).

4173 번 빌드 화면에서 로그인하고 글 목록·상세와 수정·삭제 버튼 표시를 확인했고(B03, B05), 같은 /api 프록시로 8000 번 서버에 연결됩니다. 서버를 Ctrl+C 로 껐다 켠 뒤에도 Bob 이 쓴 글이 그대로 남았습니다(R02→R04, T15). 또한 GitHub 에서 새 폴더(D:\clone-test)로 clone 해 README 의 순서대로 npm ci·.env 준비·빌드·실행했고(N01~N03), 그 폴더에서 검사 프로그램을 돌려 40 개 항목이 통과했습니다(N04). 새 DB 로 시작해 게시글이 0 개였고(N03a) 검사 때 글 번호도 작게 나왔습니다(PUT /posts/3). 첫 시도에서는 예전 폴더의 서버(8000 번)가 아직 켜져 있어 예전 글이 보였고, 그 캡처는 새 폴더 증거로 쓸 수 없어 버린 뒤 서버를 모두 끄고 다시 했습니다.

# 핵심 코드 설명

## 3 1 로그인과 JWT 발급

**파일 / 함수** backend-express/src/auth.js 설정부(6~12 행)·issueToken(14~23 행), backend-express/src/app.js 로그인 라우트(45~60 행)

### 코드 발췌

auth.js

```
const secret = process.env.JWT_SECRET ?? "";
if (!/^[0-9a-f]{64}$/i.test(secret))
  throw new Error("JWT_SECRET 은 64 자리 16 진수여야 합니다. .env 를 확인하세요.");
const key = Buffer.from(secret, "hex");
const issuer = process.env.JWT_ISSUER ?? "webservice-class";
const audience = process.env.JWT_AUDIENCE ?? "board-api";
const ttl = Number(process.env.TOKEN_TTL || 90);

// ...
export function issueToken(user) {
  // BE-01: HS256 으로 서명한 JWT 를 발급한다. (sub, exp, iss, aud 포함)
  return jwt.sign({}, key, {
    algorithm: "HS256",
    subject: String(user.id),
    issuer,
    audience,
    expiresIn: ttl,
  });
}
```

app.js (제공 코드, 로그인 라우트)

```
app.post("/api/auth/login", (req, res) => {
  const input = fields(req.body, { email: 254, password: 128 });
  const user = verifyCredentials(input.email, input.password);
  if (!user)
    throw new ApiError(
      401,
      "INVALID_CREDENTIALS",
      "이메일 또는 비밀번호를 확인하세요.",
    );
  const accessToken = issueToken(user);
  res.json({
    accessToken,
    expiresIn: Number(process.env.TOKEN_TTL || 90),
    user,
  });
});
```

### 동작과 응답 코드 설명

입력: 이메일과 비밀번호(JSON). 처리: app.js 가 입력 형식을 검사하고 users.js 의 verifyCredentials 로 계정을 확인한 뒤, 맞으면 issueToken(user)을 부릅니다. issueToken 은 서버만 아는 키(.env 의 JWT_SECRET)로 서명한 HS256 JWT 를 만들고, 안에 sub(사용자 id 를 문자열로), iss(webservice-class), aud(board-api), exp(지금부터 TOKEN_TTL 초, 기본 90 초)를 넣습니다. 비밀번호나 해시는 토큰에 넣지 않았습니다(그림 C05 에서 payload 확인). 서버는 시작할 때 JWT_SECRET 이 64 자리 16 진수인지 검사해서 비었거나 예시 글자 그대로이면 오류를 내며 켜지지 않습니다(C06). 출력과 실패: 성공 200 {accessToken, expiresIn:90, user}, 계정이 틀리면 401 INVALID_CREDENTIALS(토큰 없음, T03), 입력 형식이 틀리면 400 BAD_INPUT. 처음에는 issueToken 이 객체를 돌려줘서 응답에 accessToken 이 이중으로 들어가는 문제가 있었고, 문자열만 돌려주도록 고쳤습니다(5 절 사례 1).

## 3 2 브라우저 토큰 저장과 전송

**파일 / 함수** frontend-react/src/api.js authorizationHeaders(17~21 행)·request 의 401 처리(31~34 행)·login(47~55 행)

### 코드 발췌

api.js

```
function authorizationHeaders() {
  // FE-02: 토큰이 있을 때만 Bearer 헤더를 보낸다. (없으면 헤더를 아예 안 보냄)
  const token = session.accessToken;
  return token ? { Authorization: `Bearer ${token}` } : {};
}
// ...
  const data = response.status === 204 ? null : await response.json();
  if (!response.ok) {
    // FE-03: 401 이면 메모리의 로그인 상태도 해제한다. (403 은 로그아웃하지 않음)
    if (response.status === 401) clearSession();
// ...
export async function login(email, password) {
  // FE-01: 로그인 요청 → 받은 토큰과 사용자 정보를 메모리(session)에만 저장
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}
```

### 동작과 응답 코드 설명

로그인에 성공하면 login()이 응답의 accessToken 과 user 를 api.js 안의 session 변수(메모리)에만 저장합니다. localStorage·sessionStorage 는 쓰지 않아서 새로고침하면 로그인 상태가 사라집니다(T16). 이후 모든 요청은 authorizationHeaders()가 만든 Authorization: Bearer <토큰> 헤더를 붙여 보내고, 토큰이 없으면 헤더를 아예 보내지 않아 'Bearer undefined'가 나가지 않습니다(F06 헤더 확인, T11 에서 로그아웃 후 헤더 없음). 로그아웃 버튼은 clearSession()으로 메모리를 비웁니다. 서버가 401 을 돌려주면 request()가 clearSession()을 불러 화면의 로그인 상태도 해제하고, 오류 code(TOKEN_REQUIRED·INVALID_TOKEN·TOKEN_EXPIRED·INVALID_CREDENTIALS)마다 다른 안내 문구를 보여 줍니다(F08, F12, F13). 403 은 로그아웃하지 않고 권한 안내만 보여 주며, 204(삭제)는 본문이 없어서 JSON 으로 읽지 않습니다(F10 에서 삭제 후 오류 없음).

# 서버 인증과 작성자 권한

## 3 3 서버 JWT 검증

**파일 / 함수** backend-express/src/auth.js readBearer(26~29 행)·validClaims(32~41 행)·currentUserId(43~63 행)

### 코드 발췌

auth.js

```
function readBearer(req) {
  const match = /^Bearer ([^\s]+)$/.exec(req.get("Authorization") ?? "");
  return match ? match[1] : null;
}
// ...
export function currentUserId(req) {
  // BE-02: 토큰을 꺼내 검증하고 로그인한 사용자 id 를 돌려준다.
  // 토큰이 없거나 변조·만료되면 401 이다.
  const token = readBearer(req);
  if (!token) throw new ApiError(401, "TOKEN_REQUIRED", "로그인이 필요합니다.");
  let p;
  try {
    p = jwt.verify(token, key, { algorithms: ["HS256"], issuer, audience });
  } catch (error) {
    if (error.name === "TokenExpiredError")
      throw new ApiError(
        401,
        "TOKEN_EXPIRED",
        "로그인 시간이 만료되었습니다. 다시 로그인하세요.",
      );
    throw new ApiError(401, "INVALID_TOKEN", "토큰이 올바르지 않습니다.");
  }
  if (!validClaims(p) || !findPublic(Number(p.sub)))
    throw new ApiError(401, "INVALID_TOKEN", "토큰이 올바르지 않습니다.");
  return Number(p.sub);
}
```

### 동작과 응답 코드 설명

입력: 요청의 Authorization 헤더. 처리: readBearer 가 'Bearer 토큰' 모양에서 토큰만 꺼냅니다(없거나 모양이 다르면 null → 401 TOKEN_REQUIRED). jwt.verify 는 서명, 알고리즘(HS256 만 허용), iss·aud, 만료(exp)를 한꺼번에 확인합니다. 만료만 걸리면 401 TOKEN_EXPIRED, 서명 변조·알고리즘·iss·aud 오류는 401 INVALID_TOKEN 입니다. verify 를 통과해도 validClaims 로 sub 가 양의 정수 문자열인지, exp 가 정수인지 한 번 더 확인하고 findPublic 으로 실제 있는 사용자인지 확인합니다(아니면 401 INVALID_TOKEN). 출력: 모두 통과하면 사용자 id(숫자)를 돌려주고, app.js 의 글 등록·수정·삭제와 /api/me 가 이 id 만 믿고 씁니다. 401 응답에는 app.js 의 오류 처리기가 WWW-Authenticate: Bearer 헤더를 붙입니다(C07). 확인한 결과: 토큰 없음 401 TOKEN_REQUIRED, 서명 첫 글자를 바꾼 토큰 401 INVALID_TOKEN, TOKEN_TTL=3 으로 만료시킨 토큰 401 TOKEN_EXPIRED(GET /me·POST·PUT·DELETE 모두, A02).

## 3 4 작성자 권한 검사

**파일 / 함수** backend-express/src/auth.js assertOwner(64~72 행), app.js PUT 라우트(제공 코드), frontend-react/src/api.js canEdit(56~61 행)

## 코드 발췌

auth.js

```
export function assertOwner(userId, authorId) {
  // BE-03: 글쓴이가 아니면 403 FORBIDDEN.
  if (userId !== authorId)
    throw new ApiError(
      403,
      "FORBIDDEN",
      "본인이 작성한 글만 수정하거나 삭제할 수 있습니다.",
    );
}
```

app.js (제공 코드, 수정 라우트)

```
app.put("/api/posts/:id", (req, res) => {
  const userId = currentUserId(req),
    id = postId(req),
    p = getPost(id);
  assertOwner(userId, p.authorId);
  res.json(store.update(id, fields(req.body, { title: 100, content: 5000 })));
```

api.js

```
export function canEdit(post) {
  // FE-04: 로그인한 사람의 id 와 글쓴이 id 가 같을 때만 버튼을 보여 준다.
  // (화면에서만 숨기는 것이고, 진짜 검사는 서버가 한다)
  const me = getSession().user;
  return !!me && me.id === post.authorId;
}
```

## 동작과 응답 코드 설명

수정·삭제 라우트는 먼저 currentUserId(req)로 '지금 요청한 사람'의 id 를 토큰에서 얻고, getPost 로 글을 찾은 다음, assertOwner(userId, p.authorId)로 로그인한 사람의 id 와 글쓴이 id 를 비교합니다. 다르면 403 FORBIDDEN 이고 글은 바뀌지 않습니다(T07·T08, C08 의 02·03·04). 401 은 '누구인지 확인 못 함(토큰 없음·변조·만료)', 403 은 '누구인지는 알지만 권한이 없음'입니다. 처리 순서는 인증 → 글 존재 → 작성자 → 입력 검사라서, Bob 이 Alice 글에 공백 제목을 보내면 403, Alice 가 자기 글에 같은 입력을 보내면 400 BAD_INPUT 이 나옵니다(C08 의 05·06). 글을 등록할 때 작성자는 요청 본문이 아니라 토큰에서 얻은 userId 로 저장하고, 본문에 authorId·userId 를 끼워 넣으면 400 BAD_INPUT 입니다(T12). 화면의 canEdit 는 로그인 사용자 id 와 글쓴이 id 가 같을 때만 수정·삭제 버튼을 보여 주는 편의 기능일 뿐이고, 버튼을 숨겨도 서버 검사가 없으면 직접 요청으로 우회되므로 실제 방어는 assertOwner 입니다.

# 테스트 결과

**실행 환경** Windows 10.0.26300 / Node.js v24.21.0 / PowerShell 7.6.9 / Chrome 156. 직접 시험: PowerShell Invoke-RestMethod·curl.exe, 크롬 화면·개발자도구 Network. 자동 검사: python tools/check_api.py --mode final (--expiry)

**로컬 배포 확인** frontend-react 에서 npm run build 성공(B01) → npm run preview 로 4173 번 접속, /api 가 8000 번 백엔드로 연결(B03·B05). GitHub 에서 새 폴더로 clone 한 재현은 N01~N04.

**테스트 일시 및 코드 버전** 2026-10-28 11:41~15:12 (한국 시간). 인증 코드는 커밋 a5c2f28 과 같고, 마지막에 서버 시작 문구·주석·화면 제목만 바꾼 뒤(A03, B05) 자동 검사를 다시 돌려 40 개 통과를 확인했습니다. 제목을 바꾸기 전에 찍은 캡처는 화면 제목이 '우리의 기록'으로 보입니다.

**토큰 TTL** 일반 테스트 90 초(기본값), 만료 자동 검사는 TOKEN_TTL=3(PowerShell 에서 $env:TOKEN_TTL = "3"로 서버를 다시 켬)

| ID | 수행 절차와 사용자 | 예상 결과 | 실제 HTTP 와 code | 판정 | 증거 |
|---|---|---|---|---|---|
| T01 | 토큰 없이 목록과 상세 조회 (로그아웃 상태 화면, 자동 검사 public list·public detail) | 200, 공개 조회 | 200 / 해당 없음 | PASS | F03, A01 |
| T02 | Alice 로 로그인 (PowerShell, 화면) | 200, JWT 와 공개 사용자 정보, 화면 로그인 표시 | 200 / 해당 없음. accessToken 문자열, expiresIn 90, user id 1 Alice. 화면 'Alice 님 로그인 중' | PASS | C03, F02 |
| T03 | 틀린 비밀번호, 없는 이메일로 로그인 (PowerShell) | 401 INVALID_CREDENTIALS, 토큰 없음 | 두 경우 모두 401 / INVALID_CREDENTIALS, 토큰 없음 | PASS | C04, A01 |
| T04 | 토큰 없이 GET /api/me, POST /api/posts (curl.exe, 화면) | 각각 401 TOKEN_REQUIRED | 401 / TOKEN_REQUIRED (+WWW-Authenticate: Bearer). 화면 '로그인이 필요합니다' | PASS | C07, F08 |
| T05 | Alice 토큰으로 글 작성 후 /me | 201, authorId 1, /me 는 Alice | 201 / 해당 없음, authorId 1, Location /api/posts/8. /me 200 id 1 Alice | PASS | C08(01), C07, B04a |
| T06 | Alice 토큰으로 본인 글 수정·삭제 | 200, 204, DB 반영 | 수정 200(createdAt 유지, updatedAt 갱신), 삭제 204 후 재조회 404. 화면 수정·삭제(Bob 글)도 성공 | PASS | C08(07,12,13), F09, F10 |

| ID | 수행 절차와 사용자 | 예상 결과 | 실제 HTTP 와 code | 판정 | 증거 |
|---|---|---|---|---|---|
| T07 | Bob 정상 토큰으로 Alice 글 수정 | 403 FORBIDDEN, 원문 유지 | 403 / FORBIDDEN, 재조회 200 에서 제목 그대로 | PASS | C08(02,04), A01 |
| T08 | Bob 정상 토큰으로 Alice 글 삭제 | 403 FORBIDDEN, 글 유지 | 403 / FORBIDDEN, 재조회 200 에서 글 유지 | PASS | C08(03,04), A01 |
| T09 | 서명 첫 글자를 바꾼 토큰으로 /me, POST, PUT, DELETE | 401 INVALID_TOKEN, 데이터 변경 없음 | 네 요청 모두 401 / INVALID_TOKEN, 이어서 재조회로 불변 확인 (/me 는 직접, 변경 요청 3종은 자동 검사) | PASS | C07(5), A01 |
| T10 | 짧은 TTL 로 만료시킨 뒤 /me 와 변경 요청, 화면 (90 초 대기) | 401 TOKEN_EXPIRED, 화면 로그인 해제·안내 | 자동(TTL=3): GET /me·POST·PUT·DELETE 모두 401 / TOKEN_EXPIRED, 데이터 불변. 화면: '로그인 시간이 지났습니다', '로그아웃 상태' | PASS | A02, F12, F13 |
| T11 | 로그인 후 로그아웃, 다음 /me 요청 (크롬 Network) | 헤더에서 토큰이 빠지고 401 TOKEN_REQUIRED | Request headers 에 Authorization 없음, 401 / TOKEN_REQUIRED, Www-Authenticate: Bearer | PASS | T11a, T11b, T11c |
| T12 | Bob 토큰으로 글 등록에 authorId:1, userId:1 추가 | 400 BAD_INPUT, 위조 글 미생성 | authorId: 400 / BAD_INPUT(직접), 자동 검사에서 authorId·userId 모두 400. 위조 글은 응답이 400 이라 만들어지지 않음 (별도 목록 비교는 하지 않음) | PASS | C08(11), A01 |
| T13 | 빈 제목·공백 본문·길이 초과 입력 | 400 BAD_INPUT, DB 변경 없음 | 공백 제목(직접), 공백 제목·공백 본문·제목 101자·내용 5001 자(자동) 모두 400 / BAD_INPUT | PASS | C08(06), A01 |
| T14 | 없는 양의 정수 ID 조회·수정·삭제, 잘못된 ID 조회 | 없는 글 404, 잘못된 ID 400 | 없는 글 GET·PUT·DELETE 404 / NOT_FOUND, ID 'abc' 400 / BAD_INPUT | PASS | C08(09), A01 |
| T15 | 글 작성 후 백엔드를 Ctrl+C 로 끄고 다시 실행 (직접) | 기존 글과 작성자·내용 유지 | 재시작 후 새로고침에서 Bob 글 '재시작테스트' 그대로(200) | PASS | R02, R03, R04 |

| ID | 수행 절차와 사용자 | 예상 결과 | 실제 HTTP 와 code | 판정 | 증거 |
|---|---|---|---|---|---|
| T16 | 로그인 후 페이지 새로고침 (F5) | 공개 글 유지, 로그인 상태 해제 | 목록 유지, '로그아웃 상태' | PASS | F11 |
| T17 | 토큰 없는 수정·삭제와 Bob 의 작성·수정·삭제 | 무토큰 401, Bob 본인 작업 성공 | 무토큰 PUT·DELETE 401 / TOKEN_REQUIRED, Bob 작성 201·수정 200·삭제 204 | PASS | C08(08,10,14~16), A01, F07, F09, F10 |

표의 증거 칸 ID 는 docs/evidence/ 폴더의 파일명 앞부분과 같습니다(예: C08 = C08_be03_owner_checks_PASS.png, 괄호는 그 화면의 번호 줄). A 시리즈는 교수 제공 검사 프로그램(자동), 나머지는 제가 직접 PowerShell·curl.exe·크롬으로 한 시험(수동)입니다. 시험 명령은 AI 가 제안했고 제가 직접 실행했습니다.

# HTTP 응답 확인 기록

| 확인 항목 | 기대 응답 | 실제 HTTP 와 code | 증거 위치 |
|---|---|---|---|
| 정상 조회·로그인·수정 | **200** | 200 / 해당 없음 (로그인, /me, 본인 수정) | C03, C07, C08(07,15) |
| 새 글 등록 | **201 및 Location** | 201, Location: /api/posts/8, 응답 본문에 authorId 1 | C08(01,14) |
| 본인 글 삭제 | **204 및 빈 본문** | 204, 본문 없음 (PowerShell 이 빈 본문을 System.Byte[]로 표시). 화면은 JSON 오류 없이 목록 갱신 | C08(12,16), F10 |
| 입력 오류·추가 작성자 필드 | **400 BAD_INPUT** | 400 / BAD_INPUT (공백 제목, authorId 끼워넣기, 잘못된 ID) | C08(06,11), A01 |
| 잘못된 계정 | **401 INVALID_CREDENTIALS** | 401 / INVALID_CREDENTIALS (틀린 비밀번호, 없는 이메일) | C04 |
| 토큰 없음·헤더 형식 오류 | **401 TOKEN_REQUIRED** | 401 / TOKEN_REQUIRED (토큰 없음) | C07(1,2), C08(08,10), T11b |
| 변조·클레임 오류 | **401 INVALID_TOKEN** | 401 / INVALID_TOKEN (서명 변조) | C07(5), A01 |
| 정상 서명의 만료 토큰 | **401 TOKEN_EXPIRED** | 401 / TOKEN_EXPIRED (TTL=3 만료, 화면 90초 대기) | A02, F12, F13 |
| 다른 작성자의 변경 | **403 FORBIDDEN** | 403 / FORBIDDEN (Bob 이 Alice 글 수정·삭제) | C08(02,03,05), A01 |
| 없는 글 | **404 NOT_FOUND** | 404 / NOT_FOUND (없는 글, 삭제 후 조회) | C08(09,13), A01 |

보호 API 의 401 에는 WWW-Authenticate: Bearer 헤더가 붙었습니다. curl.exe -i 로 토큰 없는 GET /api/me·POST /api/posts 와 변조 토큰에서 확인했고(C07), 토큰 없는 수정·삭제(C08 의 08·10)에서도 나왔으며 크롬 Network 의 Response headers 에서도 'Www-Authenticate: Bearer'가 보였습니다(T11a).

401 과 403 의 화면 차이: 401(토큰 없음·만료)이 오면 화면이 '로그아웃 상태'로 바뀌고 로그인을 다시 하라는 안내가 나옵니다(F08, F12, F13). 403 은 화면에서 남의 글의 수정·삭제 버튼이 숨겨져 있어 직접 만들지 못했고, API 로만 확

인했습니다(C08, A01). 코드(api.js request)는 401 일 때만 로그인 상태를 지우므로 403 에서는 로그인이 유지되지만, 이 부분을 화면에서 직접 재현해 보지는 않았습니다.

실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인했습니다. C08 의 04 는 Bob 의 수정(02)·삭제(03)가 403 으로 막힌 뒤 GET 으로 재조회한 결과 제목 'test post'가 그대로였고, 자동 검사(A01)의 'failed request leaves data unchanged'가 무토큰·Bob·변조 토큰 경우마다 PASS 였습니다.

토큰 없이 없는 글을 수정하면 401 / TOKEN_REQUIRED(C08 의 10), 정상 토큰으로 같은 없는 글(999999)을 수정하면 404 / NOT_FOUND(C08 의 09)였습니다. 인증이 글 존재 확인보다 먼저라는 순서가 그대로 나왔습니다.

# 테스트 증거

## 로그인과 토큰 전송

로그인 성공(PowerShell 과 화면)과, 로그인한 뒤 /me 요청에 Authorization: Bearer 헤더가 실려 가는 모습입니다. 토큰 글자는 가렸습니다(원문은 제출하지 않음).

```
PowerShell 7.6.9
PS C:\Windows\System32> $body = '{"email":"alice@class.test","password":"Classroom!2026"}'
PS C:\Windows\System32> $res = Invoke-WebRequest -Method Post -Uri http://127.0.0.1:8000/api/auth/login -ContentType "application/json" -Body $body -SkipHttpErrorCheck
PS C:\Windows\System32> $res.StatusCode
200
PS C:\Windows\System32> $r = $res.Content | ConvertFrom-Json
PS C:\Windows\System32> $r.accessToken.Substring(0,25) + "..."
eyJhbGciOiJIUzI1NiIsInR5c...
PS C:\Windows\System32> $r.expiresIn
90
PS C:\Windows\System32> $r.user

id name  email
-- ----  -----
 1 Alice alice@class.test

PS C:\Windows\System32>
```

그림 2. T02 로그인 200, 토큰은 앞 25 자만 출력, expiresIn 90, user Alice (C03)

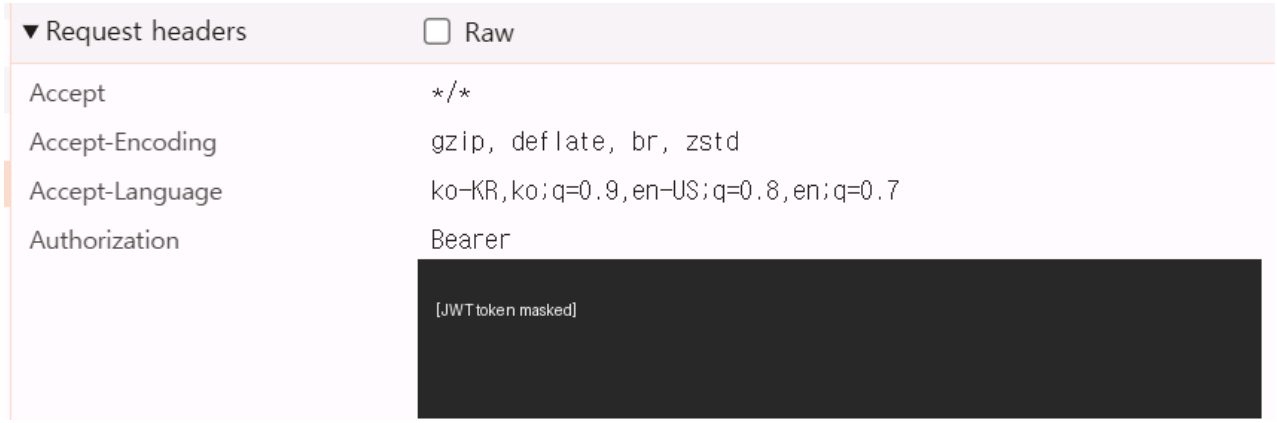



그림 3. T02·FE-02 로그인 후 /me 요청의 Authorization 헤더, 토큰 가림 (F06)

```
PS C:\Windows\System32> $body = '{"email":"alice@class.test","password":"Classroom!2026"}'
PS C:\Windows\System32> $r = Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/api/auth/login -ContentType "application/json" -Body $body
PS C:\Windows\System32> node -e "console.log(JSON.parse(Buffer.from(process.argv[1].split('.')[1],'base64url')))" $r.accessToken
{
  iat: 1793155612,
  exp: 1793155702,
  aud: 'board-api',
  iss: 'webservice-class',
  sub: '1'
}
```

그림 4. 토큰 payload: iss, aud, sub, exp(iat+90 초), 비밀번호 없음 (C05)

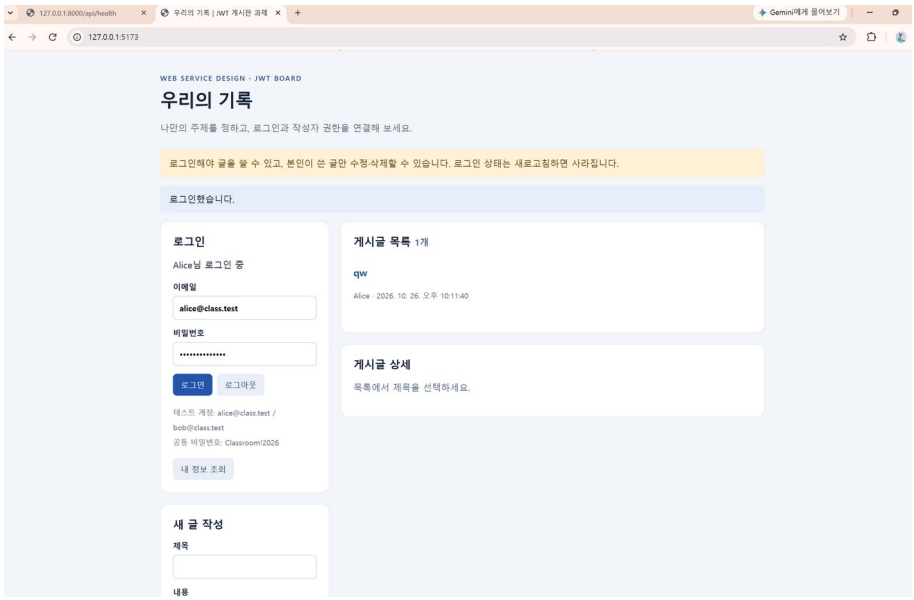



그림 5. 화면 로그인 성공 'Alice 님 로그인 중' (F02, 제목 변경 전 화면)

## 다른 작성자의 수정과 삭제

T07·T08: Bob 의 정상 토큰으로 Alice 글을 수정·삭제하면 403 FORBIDDEN 이고 글은 그대로입니다(02, 03, 04). 같은 화면에서 인증 순서(05·06·08·09·10)와 authorId 위조(11), Bob 본인 작업(14~16)도 확인했습니다.

```
PS C:\Windows\System32> $a = Login "alice@class.test"
PS C:\Windows\System32> $b = Login "bob@class.test"
PS C:\Windows\System32> $r = Req "01 Alice 글 등록" POST "/api/posts" $a '{"title":"test post","content":"written by Alice"}'
01 Alice 글 등록 -> 201 {"id":8,"title":"test post","content":"written by Alice","authorId":1,"authorName":"Alice","createdAt":"2026-10-28T03:30:05.094Z","updatedAt":"2026-10
-28T03:30:05.094Z"} [Location: /api/posts/8]
PS C:\Windows\System32> $id = ($r.Content | ConvertFrom-Json).id
PS C:\Windows\System32> Req "02 Bob이 Alice 글 수정" PUT "/api/posts/$id" $b '{"title":"hacked","content":"changed"}' | Out-Null
02 Bob이 Alice 글 수정 -> 403 {"code":"FORBIDDEN","message":"본인이 작성한 글만 수정하거나 삭제할 수 있습니다."}                    ]
PS C:\Windows\System32> Req "03 Bob이 Alice 글 삭제" DELETE "/api/posts/$id" $b $null | Out-Null
03 Bob이 Alice 글 삭제 -> 403 {"code":"FORBIDDEN","message":"본인이 작성한 글만 수정하거나 삭제할 수 있습니다."}                    ]
PS C:\Windows\System32> Req "04 글 다시 조회(안 바뀌어야 함)" GET "/api/posts/$id" $null $null | Out-Null
04 글 다시 조회(안 바뀌어야 함) -> 200 {"id":8,"title":"test post","content":"written by Alice","authorId":1,"authorName":"Alice","createdAt":"2026-10-28T03:30:05.094Z","upda
tedAt":"2026-10-28T03:30:05.094Z"}
PS C:\Windows\System32> Req "05 Bob이 공백 제목으로 Alice 글 수정" PUT "/api/posts/$id" $b '{"title":" ","content":"x"}' | Out-Null
05 Bob이 공백 제목으로 Alice 글 수정 -> 403 {"code":"FORBIDDEN","message":"본인이 작성한 글만 수정하거나 삭제할 수 있습니다."}    ]
PS C:\Windows\System32> Req "06 Alice가 공백 제목으로 자기 글 수정" PUT "/api/posts/$id" $a '{"title":" ","content":"x"}' | Out-Null
06 Alice가 공백 제목으로 자기 글 수정 -> 400 {"code":"BAD_INPUT","message":"title 입력을 확인하세요."}                        ]
PS C:\Windows\System32> Req "07 Alice가 자기 글 수정" PUT "/api/posts/$id" $a '{"title":"edited","content":"edited by Alice"}' | Out-Null
07 Alice가 자기 글 수정 -> 200 {"id":8,"title":"edited","content":"edited by Alice","authorId":1,"authorName":"Alice","createdAt":"2026-10-28T03:30:05.094Z","updatedAt":"2026
-10-28T03:30:05.315Z"}
PS C:\Windows\System32> Req "08 토큰 없이 Alice 글 수정" PUT "/api/posts/$id" $null '{"title":"x","content":"x"}' | Out-Null
08 토큰 없이 Alice 글 수정 -> 401 {"code":"TOKEN_REQUIRED","message":"로그인이 필요합니다."} [WWW-Authenticate: Bearer]          ]
PS C:\Windows\System32> Req "09 정상 토큰으로 없는 글 수정" PUT "/api/posts/999999" $a '{"title":"x","content":"x"}' | Out-Null
09 정상 토큰으로 없는 글 수정 -> 404 {"code":"NOT_FOUND","message":"게시글이 없습니다."}                                    ]
PS C:\Windows\System32> Req "10 토큰 없이 없는 글 수정" PUT "/api/posts/999999" $null '{"title":"x","content":"x"}' | Out-Null
10 토큰 없이 없는 글 수정 -> 401 {"code":"TOKEN_REQUIRED","message":"로그인이 필요합니다."} [WWW-Authenticate: Bearer]          ]
PS C:\Windows\System32> Req "11 Bob 글 등록에 authorId 끼워넣기" POST "/api/posts" $b '{"title":"x","content":"y","authorId":1}' | Out-Null
11 Bob 글 등록에 authorId 끼워넣기 -> 400 {"code":"BAD_INPUT","message":"허용된 JSON 필드만 보내세요."}                       ]
PS C:\Windows\System32> Req "12 Alice가 자기 글 삭제" DELETE "/api/posts/$id" $a $null | Out-Null
12 Alice가 자기 글 삭제 -> 204 System.Byte[]                                                                              ]
PS C:\Windows\System32> Req "13 삭제 후 조회" GET "/api/posts/$id" $null $null | Out-Null
13 삭제 후 조회 -> 404 {"code":"NOT_FOUND","message":"게시글이 없습니다."}                                                  ]
PS C:\Windows\System32> $r2 = Req "14 Bob 글 등록" POST "/api/posts" $b '{"title":"bob post","content":"written by Bob"}'
14 Bob 글 등록 -> 201 {"id":9,"title":"bob post","content":"written by Bob","authorId":2,"authorName":"Bob","createdAt":"2026-10-28T03:30:05.515Z","updatedAt":"2026-10-28T03:
30:05.515Z"} [Location: /api/posts/9]
PS C:\Windows\System32> $id2 = ($r2.Content | ConvertFrom-Json).id
PS C:\Windows\System32> Req "15 Bob이 자기 글 수정" PUT "/api/posts/$id2" $b '{"title":"bob edited","content":"ok"}' | Out-Null
15 Bob이 자기 글 수정 -> 200 {"id":9,"title":"bob edited","content":"ok","authorId":2,"authorName":"Bob","createdAt":"2026-10-28T03:30:05.515Z","updatedAt":"2026-10-28T03:30:
05.561Z"}
PS C:\Windows\System32> Req "16 Bob이 자기 글 삭제" DELETE "/api/posts/$id2" $b $null | Out-Null
16 Bob이 자기 글 삭제 -> 204 System.Byte[]                                                                                ]
PS C:\Windows\System32>
```

그림 6. T05~T08, T12, T14, T17 직접 시험 16 개 요청 (C08 3 차, 모두 기대대로)

## 토큰 변조와 만료

T09: 서명 첫 글자를 바꾼 토큰은 401 INVALID_TOKEN 입니다(변경 요청 3 종은 자동 검사 A01). T10: 토큰이 만료되면 서버가 401 TOKEN_EXPIRED 를 주고, 화면은 로그인 상태를 지우고 안내를 보여 줍니다.

```
PS C:\Windows\System32> [Console]::OutputEncoding = [System.Text.Encoding]::UTF8
PS C:\Windows\System32> $u = "http://127.0.0.1:8000"
PS C:\Windows\System32> $j = "application/json"
PS C:\Windows\System32> $f = '^HTTP|^WWW-Authenticate|^\{'
PS C:\Windows\System32> $a = (Invoke-RestMethod -Method Post -Uri "$u/api/auth/login" -ContentType $j -Body '{"email":"alice@class.test","password":"Classroom!2026"}').access
Token
PS C:\Windows\System32> $b = (Invoke-RestMethod -Method Post -Uri "$u/api/auth/login" -ContentType $j -Body '{"email":"bob@class.test","password":"Classroom!2026"}').accessTo
ken
PS C:\Windows\System32> $p = $a.Split(".")
PS C:\Windows\System32> $x = if ($p[2][0] -eq 'A') { 'B' } else { 'A' }
PS C:\Windows\System32> $bad = $p[0] + "." + $p[1] + "." + $x + $p[2].Substring(1)
PS C:\Windows\System32> "--- 1) 토큰 없이 GET /api/me"
--- 1) 토큰 없이 GET /api/me
PS C:\Windows\System32> curl.exe -s -i "$u/api/me" | Select-String $f

HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer
{"code":"TOKEN_REQUIRED","message":"로그인이 필요합니다."}

PS C:\Windows\System32> "--- 2) 토큰 없이 POST /api/posts"
--- 2) 토큰 없이 POST /api/posts
PS C:\Windows\System32> curl.exe -s -i -X POST "$u/api/posts" | Select-String $f

HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer
{"code":"TOKEN_REQUIRED","message":"로그인이 필요합니다."}

PS C:\Windows\System32> "--- 3) Alice 토큰으로 GET /api/me"
--- 3) Alice 토큰으로 GET /api/me
PS C:\Windows\System32> curl.exe -s -i -H "Authorization: Bearer $a" "$u/api/me" | Select-String $f

HTTP/1.1 200 OK
{"id":1,"name":"Alice","email":"alice@class.test"}

PS C:\Windows\System32> "--- 4) Bob 토큰으로 GET /api/me"
--- 4) Bob 토큰으로 GET /api/me
PS C:\Windows\System32> curl.exe -s -i -H "Authorization: Bearer $b" "$u/api/me" | Select-String $f

HTTP/1.1 200 OK
{"id":2,"name":"Bob","email":"bob@class.test"}

PS C:\Windows\System32> "--- 5) 서명 첫 글자를 바꾼 토큰으로 GET /api/me"
--- 5) 서명 첫 글자를 바꾼 토큰으로 GET /api/me
PS C:\Windows\System32> curl.exe -s -i -H "Authorization: Bearer $bad" "$u/api/me" | Select-String $f

HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer
{"code":"INVALID_TOKEN","message":"토큰이 올바르지 않습니다."}
```

그림 7. T04·T09 토큰 없음 401, 정상 토큰 200, 서명 변조 401 INVALID_TOKEN (C07)

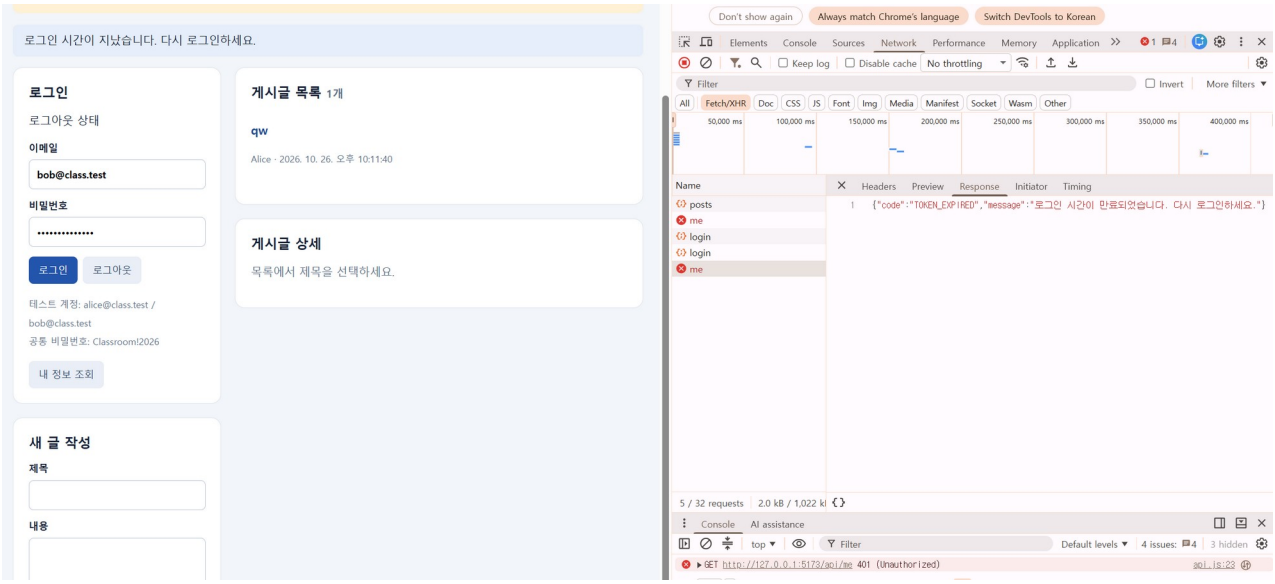



그림 8. T10 로그인 후 약 100 초 뒤 '내 정보 조회': 401 TOKEN_EXPIRED, 로그아웃 상태 (F12)

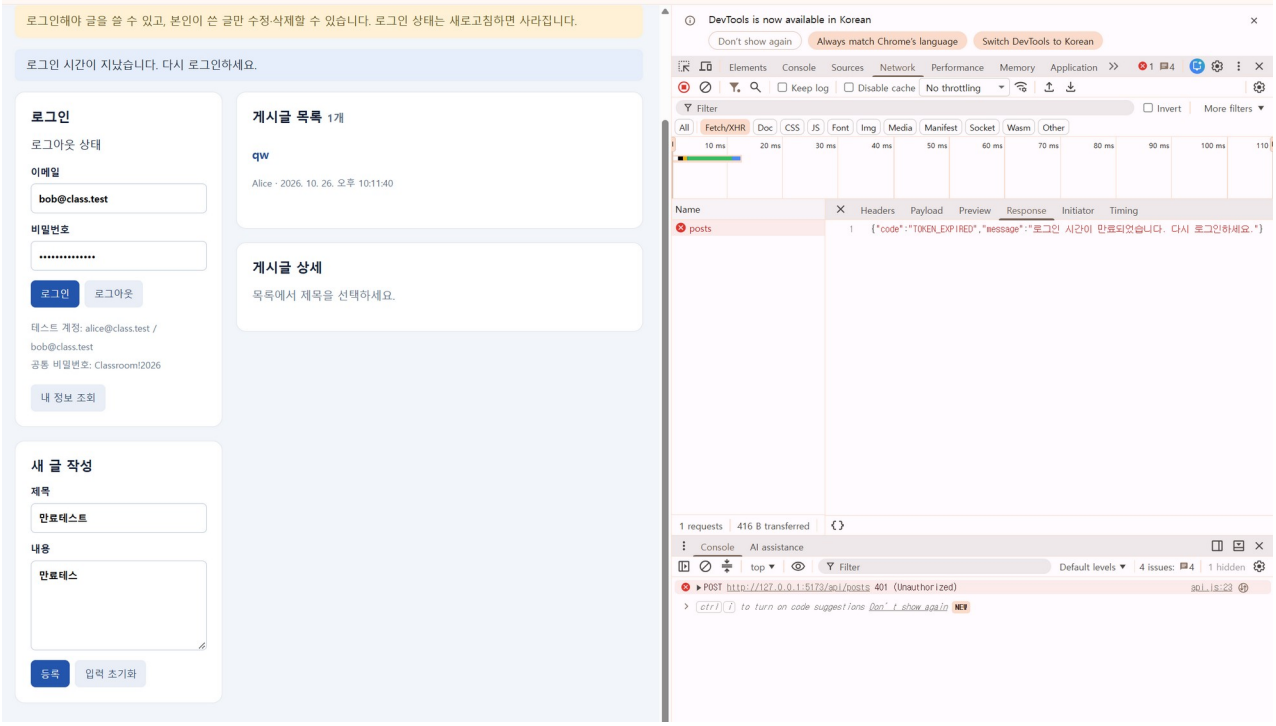



그림 9. T10 같은 방식으로 글 등록: POST 401 TOKEN_EXPIRED, 글 안 생김 (F13)

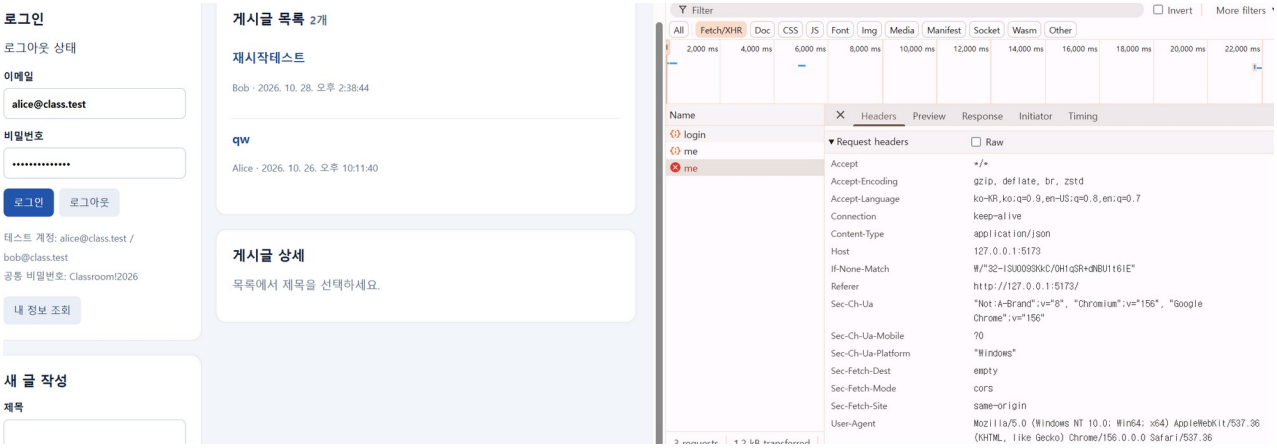



그림 10. T11 로그아웃 후 /me 의 Request headers 에 Authorization 없음 (T11c)

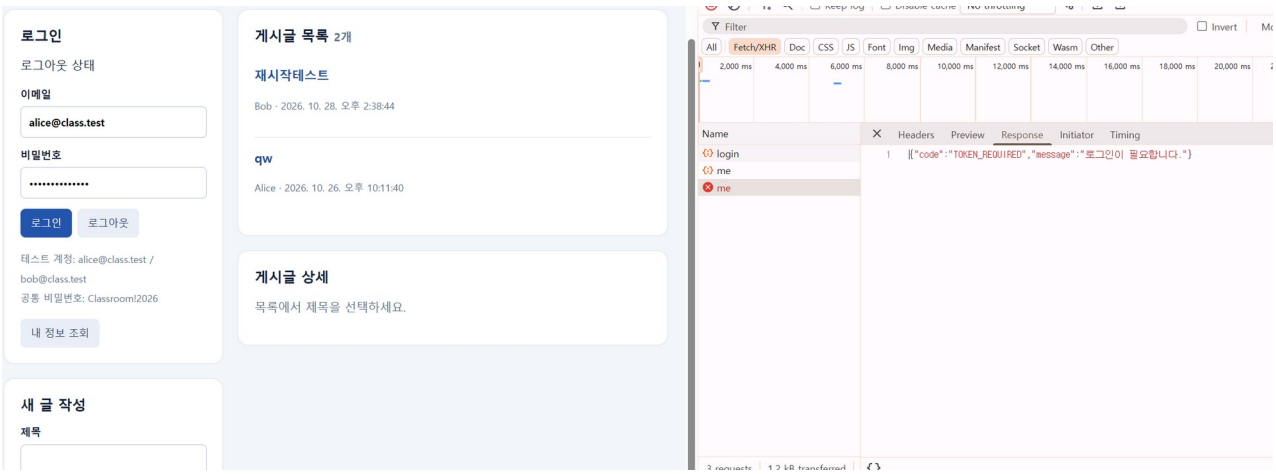



그림 11. T11 같은 요청의 Response: 401 TOKEN_REQUIRED (T11b)

## 자동 검사와 로컬 배포 증거

자동 검사 프로그램은 교수 제공 tools/check_api.py 이며 실행 중인 서버에 요청을 보내 PASS/FAIL 을 출력합니다. python tools/check_api.py --mode final 은 40 개, TOKEN_TTL=3 으로 서버를 켠 뒤 --expiry 를 붙이면 만료

GET · POST · PUT · DELETE 까지 48 개가 통과했습니다. 화면(UI)과 서버 재시작은 이 프로그램이 대신하지 않아서 직접 시험했습니다.

```
PS C:\Windows\System32> cd D:\JWT-Assignment
PS D:\JWT-Assignment> python tools/check_api.py --mode final
PASS health
PASS public list
PASS bad ID
PASS missing ID
PASS wrong password
PASS unknown email
PASS no token me
PASS no token create
PASS login alice@class.test
PASS login bob@class.test
PASS Alice me
PASS Bob me
PASS create post
PASS public detail
PASS no token PUT
PASS Bob denied PUT
PASS failed request leaves data unchanged
PASS no token DELETE
PASS Bob denied DELETE
PASS failed request leaves data unchanged
PASS reject authorId
PASS reject userId
PASS input validation
PASS input validation
PASS input validation
PASS input validation
PASS tampered GET /me
PASS tampered POST /posts
PASS tampered PUT /posts/11
PASS tampered DELETE /posts/11
PASS failed request leaves data unchanged
PASS owner update
PASS create post
PASS Bob own update
PASS missing protected PUT
PASS missing protected DELETE
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 40 checks passed (final). UI and restart tests must be performed separately.
```

그림 12. python tools/check_api.py --mode final: 40 개 PASS (A01)

```
PASS reject userId
PASS input validation
PASS input validation
PASS input validation
PASS input validation
PASS tampered GET /me
PASS tampered POST /posts
PASS tampered PUT /posts/13
PASS tampered DELETE /posts/13
PASS failed request leaves data unchanged
PASS owner update
PASS create post
PASS Bob own update
PASS missing protected PUT
PASS missing protected DELETE
PASS login alice@class.test
Waiting 4 seconds for expiration
PASS expired GET /me
PASS expired POST /posts
PASS expired PUT /posts/13
PASS expired DELETE /posts/13
PASS failed request leaves data unchanged
PASS login alice@class.test
PASS login bob@class.test
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 48 checks passed (final). UI and restart tests must be performed separately.
PS D:\JWT-Assignment>
```

그림 13. --expiry(TOKEN_TTL=3): 48 개 PASS, expired 4 종 포함 (A02)

로컬 배포: 빌드 성공 로그(B01), 4173 번 preview 실행(B02), 4173 번 화면에서 로그인과 버튼 표시(B03, B05), 서버 재시작 뒤 글 유지(R02~R04)입니다. 새 폴더 재현은 N01~N04 입니다.

```
PowerShell 7.6.9
PS C:\Windows\System32> cd D:\JWT-Assignment\frontend-react
PS D:\JWT-Assignment\frontend-react> npm run build

> jwt-board-react@1.0.0 build
> vite build

vite v8.3.4 building client environment for production...
✓ 16 modules transformed.
computing gzip size...
dist/index.html                   0.42 kB │ gzip:  0.31 kB
dist/assets/index-DaISom2Y.css    1.65 kB │ gzip:  0.79 kB
dist/assets/index-BnIJ0WZY.js   226.20 kB │ gzip: 71.21 kB

✓ built in 536ms
```

그림 14. npm run build 성공 (B01)

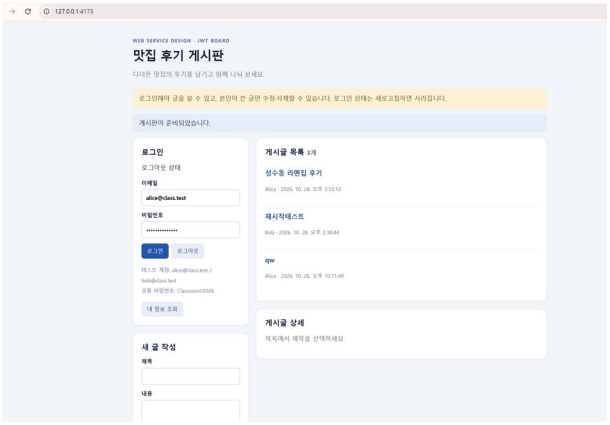



그림 15. 4173 번 빌드 화면, 새 제목 (B05)

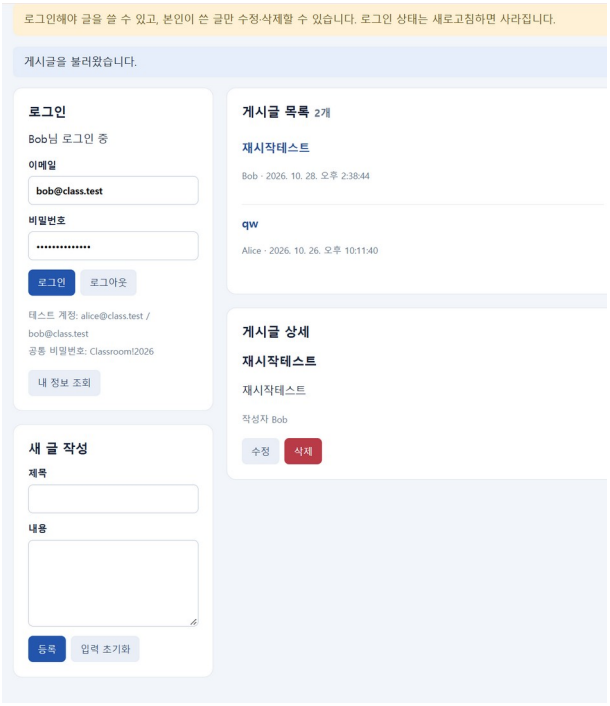



그림 16. T15 재시작 전: Bob 이 '재시작테스트' 작성 (R02)

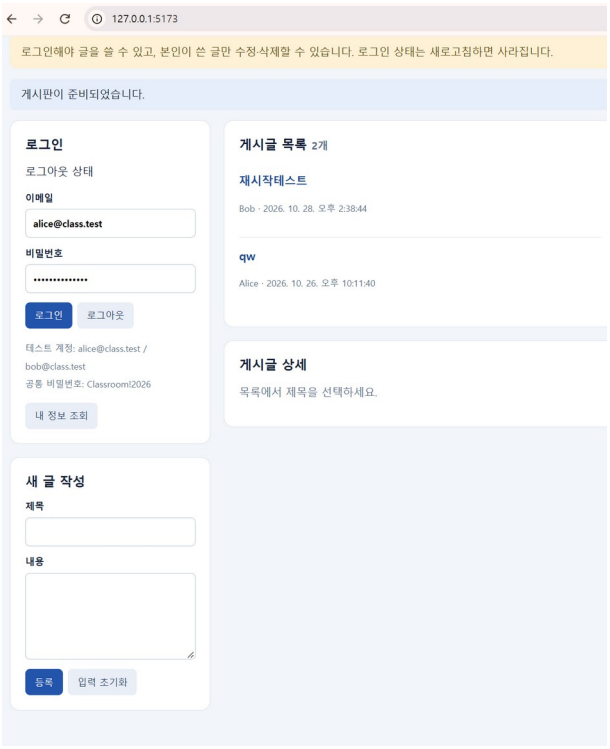



그림 17. T15 재시작 후: 글이 그대로 남음 (R04)




그림 18. 새 폴더(clone-test)에서 npm start (N01a)

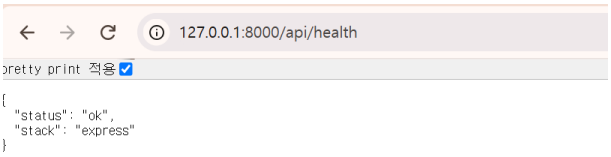



그림 19. 새 폴더 백엔드 /api/health: ok (N01b)

```
PS C:\Windows\System32> cd D:\clone-test\frontend-react
PS D:\clone-test\frontend-react> npm ci

added 19 packages, and audited 20 packages in 15s

8 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS D:\clone-test\frontend-react> npm run build

> jwt-board-react@1.0.0 build
> vite build

vite v8.3.4 building client environment for production...
✓ 16 modules transformed.
computing gzip size...
dist/index.html                   0.44 kB │ gzip:  0.31 kB
dist/assets/index-DaISom2Y.css    1.65 kB │ gzip:  0.79 kB
dist/assets/index-CloBYnSq.js   226.19 kB │ gzip: 71.20 kB

✓ built in 773ms
PS D:\clone-test\frontend-react> npm run preview

> jwt-board-react@1.0.0 preview
> vite preview --host 127.0.0.1

  ➜  Local:   http://127.0.0.1:4173/
  ➜  press h + enter to show help
```

그림 20. 새 폴더 프론트 npm ci · build · preview (N02)

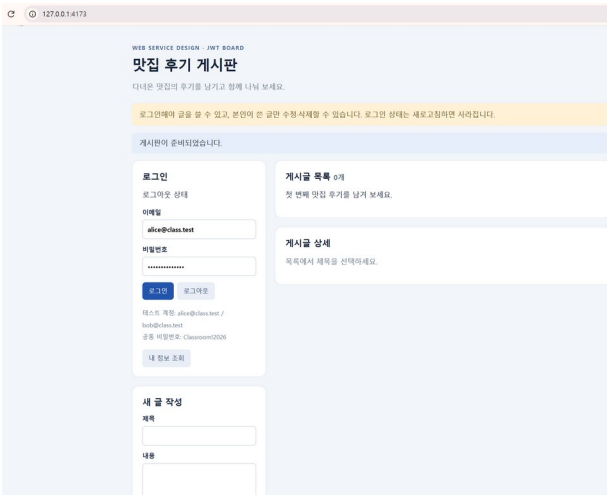



그림 21. 새 폴더 4173 번, 새 DB 라 글 0 개 (N03a)

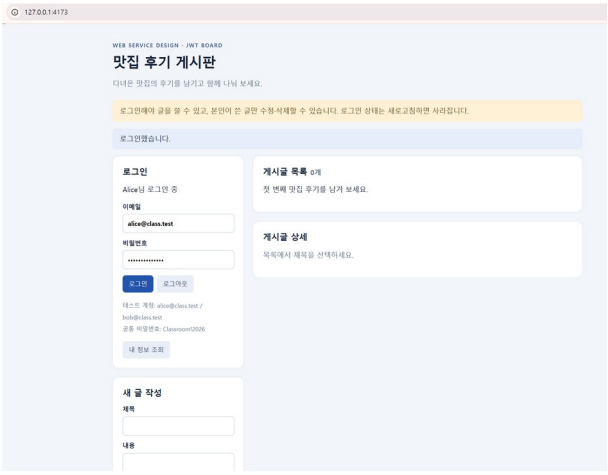



그림 22. 새 폴더 4173 번 Alice 로그인 (N03b)

```
PS C:\Windows\System32> cd D:\clone-test
PS D:\clone-test> python tools/check_api.py --mode final
PASS health
PASS public list
PASS bad ID
PASS missing ID
PASS wrong password
PASS unknown email
PASS no token me
PASS no token create
PASS login alice@class.test
PASS login bob@class.test
PASS Alice me
PASS Bob me
PASS create post
PASS public detail
PASS no token PUT
PASS Bob denied PUT
PASS failed request leaves data unchanged
PASS no token DELETE
PASS Bob denied DELETE
PASS failed request leaves data unchanged
PASS reject authorId
PASS reject userId
PASS input validation
PASS input validation
PASS input validation
PASS input validation
PASS tampered GET /me
PASS tampered POST /posts
PASS tampered PUT /posts/3
PASS tampered DELETE /posts/3
PASS failed request leaves data unchanged
PASS owner update
PASS create post
PASS Bob own update
PASS missing protected PUT
PASS missing protected DELETE
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 40 checks passed (final). UI and restart tests must be performed separately.
PS D:\clone-test>
```

그림 23. 새 폴더에서 자동 검사 40 개 PASS (N04)

# AI 활용과 참고 자료

**AI 사용 여부** 사용 (Claude, Anthropic). 구현 순서 안내, 코드 초안, 시험 명령 제안, 오류 원인 진단, README 와 보고서 초안에 사용했습니다. 제안된 코드와 명령은 제가 직접 붙여 넣어 실행하고 결과를 캡처로 확인했습니다.

| 도구와 모델 | 사용 목적 | 반영한 파일 또는 보고서 항목 | 반영 방식 |
|---|---|---|---|
| Claude (Anthropic) 코딩 도우미 세션, 세부 모델 버전은 미확인 | 학습·설계: 단계별 구현 안내, 수업 06 JWT 자료와 시작 코드를 읽고 코드 방향 제안 | auth.js, api.js, main.jsx | 그대로 반영(제가 붙여넣고 실행·검증) |
| 같은 세션 | 테스트: PowerShell·curl.exe 시험 명령 제안, 오류 원인 진단 명령 제안 | 5 절 시험 증거 C03~C08 | 제안 명령을 그대로 실행하고 결과 캡처 |
| 같은 세션 | 문장정리: README 와 이 보고서의 초안 작성 | README.md, docs/report.pdf | 수정 후 반영(결과·수치는 제 캡처와 대조) |

## 대표 사례

**도구 / 날짜** Claude / 2026-10-28 (사례 1: BE-01)

**요청한 내용** 수업 자료(06 JWT 로그인)를 참고해 issueToken 을 최대한 간단하고 직관적으로 만들어 달라고 요청

**AI 가 제안한 내용** 1 차 초안은 { accessToken, expiresIn } 객체를 반환하고 iss/aud 를 board-api/board-client 로 둔 코드. 이후 app.js 와 .env.example 을 읽고 JWT 문자열만 반환, iss=webservice-class·aud=board-api 로 다시 작성

**실제 반영 위치** backend-express/src/auth.js issueToken (이 보고서 3.1)

**내가 확인하고 수정한 내용** 직접 실행한 로그인 응답에 accessToken 이 이중으로 들어간 것을 발견해 AI 에게 알렸고, AI 초안의 반환 형태와 iss/aud 값이 제공 파일과 달랐다는 점을 확인한 뒤 정정본으로 교체했습니다.

**검증 방법과 결과** T02·T03: 로그인 200 에서 accessToken 이 문자열(C03), payload 의 iss=webservice-class, aud=board-api, sub='1', exp=iat+90(C05), 예시 키로 서버 시작 시 거부(C06)

**도구 / 날짜:** Claude / 2026-10-28 (사례 2: BE-03 작성자 검사 실패와 원인)

**요청한 내용:** assertOwner 를 구현하고 C08 의 시험 명령(Alice·Bob 요청 16 개)을 제안해 달라고 요청

**AI 가 제안한 내용:** userId !== authorId 이면 403 FORBIDDEN 을 던지는 코드, 시험 명령, 실패 시 원인을 가르는 진단 명령

**실제 반영 위치:** backend-express/src/auth.js assertOwner (이 보고서 3.4)

**내가 확인하고 수정한 내용:** 1 차·2 차 시험에서 Bob 이 Alice 글을 수정(200)하고 삭제(204)했습니다 (C08_try1_FAIL, C08_try2_FAIL). 진단하니 auth.js 의 마지막 저장 시각이 BE-02 적용 때(12:08:55)로 멈춰 있어서, assertOwner 수정을 파일에 저장하지 않았던 것이 원인이었습니다(_diag). 저장하고 서버를 다시 켰습니다. AI 의 진단 명령도 'FORBIDDEN' 글자가 TODO 주석에 있어서 잘못 '있음'으로 나오는 오탐이 있었습니다.

**검증 방법과 결과:** 3 차 시험(C08)에서 02·03·05 가 403 FORBIDDEN, 04 재조회에서 글 그대로, 06 은 400. 이후 자동 검사 A01 의 'Bob denied PUT/DELETE'도 PASS

**도구 / 날짜:** Claude / 2026-10-28 (사례 3: 프론트 FE-01~04 와 검증 한계)

**요청한 내용:** api.js 와 main.jsx 의 로그인 연결·토큰 전송·권한 표시를 수업 방식으로 간단하게 만들어 달라고 요청

**AI 가 제안한 내용:** login, authorizationHeaders, 401 처리, canEdit 와 안내 문구 코드. AI 쪽에서는 npm 설치가 막혀 브라우저로 실행하지 못하고 문법 검사와 가짜 서버 시험까지만 했다고 밝혔습니다.

**실제 반영 위치:** frontend-react/src/api.js, main.jsx (이 보고서 3.2)

**내가 확인하고 수정한 내용:** 코드를 붙여넣고 크롬에서 직접 시험했습니다. 처음 Network 에서 Authorization 헤더가 안 보여 당황했지만, 토큰이 90 초 만료로 이미 지워진 뒤 눌러서 헤더를 안 보낸 것이었고 로그인 직후에 다시 하니 헤더가 보였습니다(F06). 파일 카드가 안 열려 코드를 채팅에서 복사해 붙여넣기도 했습니다.

**검증 방법과 결과:** F01~F13: 로그인·버튼 표시·글 작성·수정·삭제·만료 안내·새로고침, 4173 빌드 화면 B03·B05

## 참고 자료

| 자료 제목과 URL | 사용한 위치 | 반영 내용 및 출처 표시 |
|---|---|---|
| 수업 자료 06 JWT 로그인 (28·35·37·38·40 쪽) | auth.js issueToken, readBearer, jwt.verify 옵션, validClaims, 만료·변조 구분 | 코드 구조와 순서를 따랐고, sub 문자열 검사와 실제 사용자 확인은 서비스 요구에 맞게 추가 |
| 과제 제공 문서 docs/02_API 명세, 03_구현 체크리스트, 05_테스트시나리오, 07_로컬실행안내 | 응답 코드·처리 순서, T01~T17, 실행 절차 | 그대로 따름 |
| 제공 코드: app.js, store.js, users.js, .env.example, tools/check_api.py | 라우트, DB, 계정, 자동 검사 | 수정 없이 사용(app.js 는 시작 문구 한 군데만 변경) |

# 이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문의 authorId 를 믿으면 Bob 이 {"authorId":1}을 끼워 넣어 Alice 이름으로 글을 쓸 수 있습니다. 제 코드는 작성자를 본문에서 받지 않고 토큰에서 얻은 userId 로 저장하고, 본문에 authorId·userId 같은 허용 안 된 필드가 있으면 400 BAD_INPUT 으로 막습니다(C08 의 11, A01 의 reject authorId·userId).

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼을 숨기는 건 화면에서만 막는 것이라서, 개발자도구나 curl 로 서버에 직접 PUT·DELETE 를 보내면 우회됩니다. 그래서 서버의 assertOwner(userId, p.authorId)가 로그인한 사람과 글쓴이를 비교해 다르면 403 으로 막습니다. 화면에서 버튼을 숨겨도 Bob 토큰으로 직접 요청했을 때 403 이 나오는 것을 확인했습니다(C08, A01).

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 토큰 안의 글자를 읽기만 해서 위조된 토큰도 그대로 읽힙니다. verify 는 서버의 비밀 키로 서명을 다시 계산해 맞는지 확인하고 만료까지 검사합니다. 제 코드는 jwt.verify 로 서명·HS256 알고리즘·iss·aud·exp 를 확인하고, 이어서 sub 가 양의 정수 문자열인지, exp 가 정수인지, 그 사용자가 실제로 있는지까지 확인합니다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 메모리의 토큰과 사용자 정보를 지워서 다음 요청부터 Authorization 헤더가 빠집니다(T11). 토큰 만료는 서버가 401 TOKEN_EXPIRED 를 주고 화면이 로그인 상태를 지웁니다(T10). 새로고침은 메모리가 사라져 로그인 상태가 해제됩니다(T16). 이미 복사된 토큰은 로그아웃해도 서버가 따로 막지 않아서 만료 시간(90 초)이 지나기 전까지는 유효합니다. 그래서 TTL 을 짧게 둡니다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

가장 어려웠던 것은 BE-03 을 만든 뒤 Bob 이 Alice 글을 수정·삭제할 수 있었던 1·2 차 실패였습니다. 처음에는 코드가 틀렸나 서버가 안 켜졌나 헷갈렸는데, 진단해 보니 auth.js 를 저장하지 않아서 예전 코드가 계속 돌고 있었습니다. 저장하고 서버를 다시 켠 뒤 같은 16 개 요청을 다시 보내 02·03·05 가 403 으로 바뀐 것을 확인했습니다(C08 3 차). 그 뒤로는 파일 저장 표시(●)와 서버 재시작을 먼저 확인합니다.

**현재 남은 문제와 다음에 개선할 점** 복사된 토큰은 만료 전까지 유효하다(서버에서 폐기하지 않음). HTTPS 없이 로컬 http 로만 시험했다. 로그인 시도 횟수 제한은 없다. 화면에서 403 이 나오는 모습은 직접 재현하지 못하고 API 로만 확인했다. 제목 변경 전에 찍은 캡처는 화면 제목이 '우리의 기록'이다.

제출 확인: [x] 실제 코드와 설명 일치 [x] 테스트 직접 수행 [x] AI 활용 및 출처 기록 [x] 비밀 키와 토큰 제거

GitHub 확인: [x] 루트 README 작성 [ ] 보고서 PDF 와 증거 업로드 [ ] README 문서 링크 확인 [x] 교수자 접근 가능(Public) [ ] 저장소 URL 과 커밋 SHA 제출 (마지막 3 개는 push 와 LMS 제출 후 확인)